Университет ИТМО
Факультет программной инженерии и компьютерной техники
Направление подготовки: 09.03.04 - Системное и прикладное программное обеспечение

Лабораторная работа №1
по Вычислительной математике
«Решение системы линейных алгебраических уравнений»
Метод простых итераций

Выполнил:
Лейковский Никита Вячеславович
Преподаватель:
Бострикова Дарья Константиновна

г. Санкт-Петербург
2026 г.

1 Цель работы и описание метода

1.1 Цель работы

Целью лабораторной работы является реализация численного метода решения системы линейных алгебраических уравнений (СЛАУ) — метода простых итераций — в виде отдельной подпрограммы с передачей данных через параметры. Работа предполагает обеспечение ввода матрицы и вектора правых частей как с клавиатуры, так и из файла, а также вывод нормы матрицы перехода, вектора неизвестных, количества затраченных итераций и вектора погрешностей.

1.2 Постановка задачи

Дана система линейных алгебраических уравнений (СЛАУ) вида

$$A\mathbf{x} = \mathbf{b},$$

где A — квадратная матрица коэффициентов размера $n \times n$ ($n \leqslant 20$), $\mathbf{b}$ — вектор правых частей, $\mathbf{x}$ — вектор неизвестных. Требуется найти приближённое решение с заданной точностью ε методом простых итераций.

1.3 Идея метода простых итераций

Метод простых итераций относится к классу итерационных методов. Для его применения исходную систему приводят к эквивалентному виду

$$\mathbf{x} = C\mathbf{x} + \mathbf{d},$$

где матрица перехода C и вектор $\mathbf{d}$ определяются следующим образом (при условии $a_{ii} \neq 0$):

$$d_i = \frac{b_i}{a_{ii}}, \quad c_{ij} = \begin{cases} 0, & i = j, \\ -\dfrac{a_{ij}}{a_{ii}}, & i \neq j. \end{cases}$$

На каждой итерации компоненты нового приближения $\mathbf{x}^{(k+1)}$ вычисляются исключительно на основе значений, полученных на предыдущем шаге $\mathbf{x}^{(k)}$. Ключевое свойство метода простых итераций: обновлённые значения не используются внутри текущей итерации. Расчётная формула имеет вид:

$$x_i^{(k+1)} = d_i + \sum_{j \neq i} c_{ij} x_j^{(k)}, \quad i = 1, 2, \ldots, n.$$

В качестве начального приближения принимается вектор $\mathbf{x}^{(0)} = \mathbf{d}$.

1.4 Критерий окончания итераций

Итерационный процесс продолжается до тех пор, пока не будет достигнута заданная точность ε. В качестве критерия остановки используется бесконечная норма разности двух последовательных приближений:

$$\max_i \left| x_i^{(k)} - x_i^{(k-1)} \right| \leqslant \varepsilon.$$

По завершении расчётов выводится вектор абсолютных погрешностей: $\left(\left| x_i^{(k)} - x_i^{(k-1)} \right| \right)_i^n$

1.5 Достаточное условие сходимости

Для гарантированной сходимости метода простых итераций достаточно выполнения условия *строгого диагонального преобладания* матрицы A:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad i = 1, \ldots, n.$$

В программе реализована проверка данного условия. Если в исходной матрице преобладание отсутствует, выполняется алгоритм перестановки строк. Если путём перестановок достичь диагонального преобладания невозможно, программа выводит сообщение о том, что сходимость метода не гарантирована.

1.6 Норма матрицы

В работе вычисляется строчная норма матрицы перехода C:

$$\|C\|_\infty = \max_i \sum_{j=1}^{n} |c_{ij}|.$$

Если $\|C\|_\infty < 1$, итерационный процесс сходится к единственному решению СЛАУ при любом начальном приближении.

1.7 Алгоритм

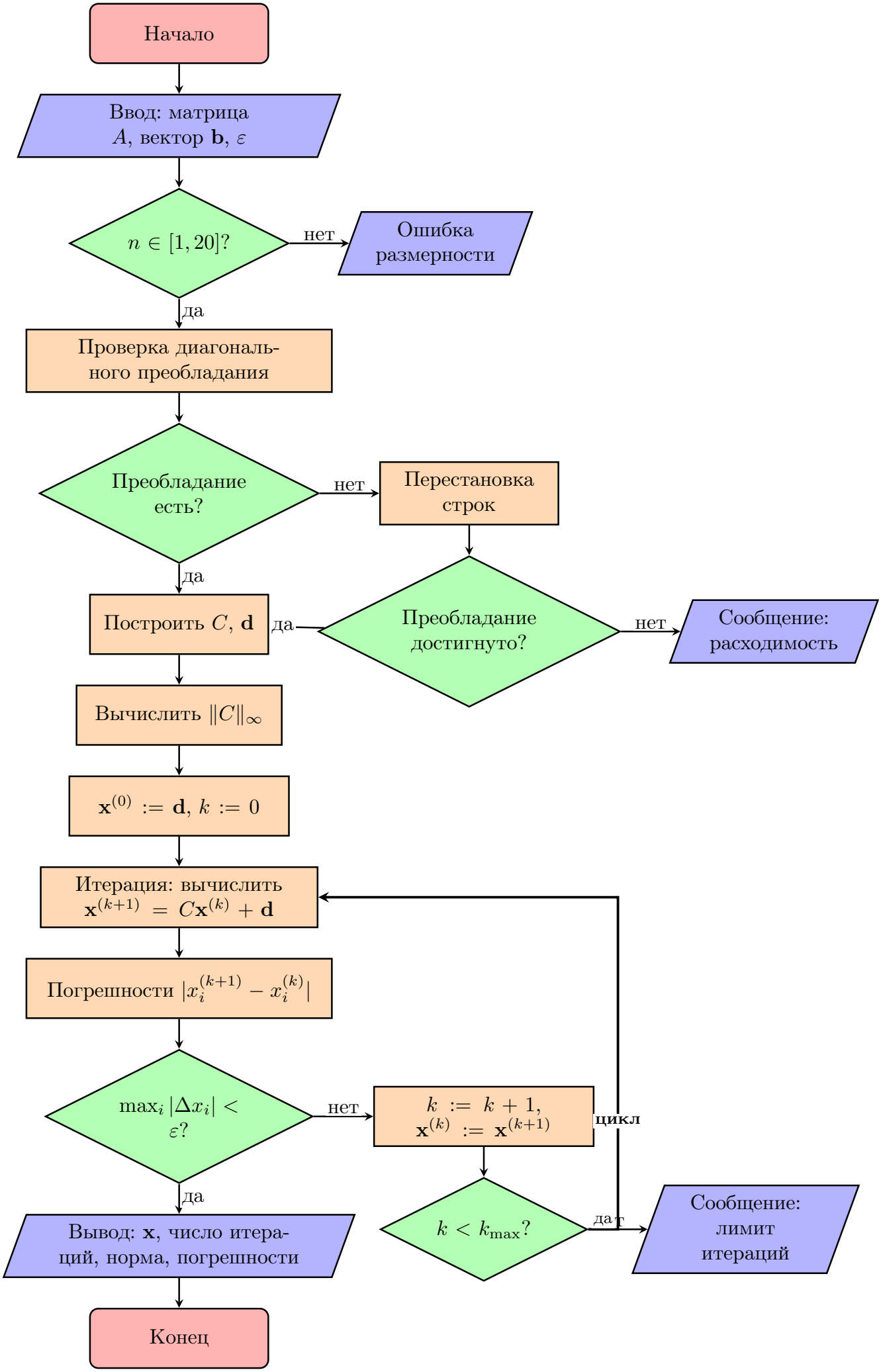
1. Проверка корректности размерности $n \in [1, 20]$.
2. Проверка и, при необходимости, обеспечение диагонального преобладания путём перестановки строк (и соответствующих элементов вектора $\mathbf{b}$).
3. Формирование расчётной матрицы C и вектора $\mathbf{d}$.
4. Вычисление нормы матрицы $\|C\|_\infty$.
5. Принятие начального приближения $\mathbf{x}^{(0)} = \mathbf{d}$.
6. Итерационный цикл:

- Вычисление значений $\mathbf{x}^{(k+1)}$ по вектору $\mathbf{x}^{(k)}$.
- Расчёт текущей погрешности для каждой компоненты.
- Проверка критерия $\max_i \Delta x_i \leqslant \varepsilon$.

7. Вывод итогового вектора неизвестных, числа итераций, нормы и вектора погрешностей.

2 Блок-схема алгоритма метода простых итераций

Схема соответствует программе: ввод матрицы A, вектора $\mathbf{b}$, точности ε; проверка размерности и диагонального преобладания; построение матриц C и вектора $\mathbf{d}$ для итерационного процесса $\mathbf{x}^{(k+1)} = C\mathbf{x}^{(k)} + \mathbf{d}$; итерации с зацикливанием при $\max_i |\Delta x_i| \geqslant \varepsilon$ и $k < k_{\max}$.


Начало
Ввод: матрица A, вектор $\mathbf{b}$, ε
$n \in [1, 20]$?
нет
Ошибка размерности
да
Проверка диагонального преобладания
Преобладание есть?
нет
Перестановка строк
Преобладание достигнуто?
да
нет
Сообщение: расходимость
да
Построить C, $\mathbf{d}$
Вычислить $\|C\|_\infty$
$\mathbf{x}^{(0)} := \mathbf{d}$, $k := 0$
Итерация: вычислить $\mathbf{x}^{(k+1)} = C\mathbf{x}^{(k)} + \mathbf{d}$
Погрешности $|x_i^{(k+1)} - x_i^{(k)}|$
$\max_i |\Delta x_i| < \varepsilon$?
нет
$k := k + 1$, $\mathbf{x}^{(k)} := \mathbf{x}^{(k+1)}$
цикл
$k < k_{\max}$?
да
Сообщение: лимит итераций
да
Вывод: $\mathbf{x}$, число итераций, норма, погрешности
Конец

3 Листинг программы

3.1 Реализация метода простых итераций

Ниже приведена реализация метода простых итераций на языке Go. Численный метод реализован в рамках структуры `IterativeSolver`, методы которой обеспечивают проверку и достижение диагонального преобладания, расчет нормы матрицы перехода и выполнение итерационного процесса. Обработка данных (валидация, чтение файлов и работа с GUI) вынесена на сторону клиента (JavaScript).

3.2 Фрагмент кода сервера (math.go)

Листинг 1: Проверка и автоматическое достижение диагонального преобладания

```go
func (s *IterativeSolver) CheckAndFixDominance() bool {
    n := s.N
    if n == 1 {
        return math.Abs(s.Matrix[0][0]) > 1e-25
    }
    p := make([]int, n)
    for i := range p {
        p[i] = -1
    }
    used := make([]bool, n)
    var findPermutation func(row int) bool
    findPermutation = func(row int) bool {
        if row == n {
            return true
        }
        for i := 0; i < n; i++ {
            if !used[i] {
                sum := 0.0
                for j := 0; j < n; j++ {
                    sum += math.Abs(s.Matrix[i][j])
                }
                diag := math.Abs(s.Matrix[i][row])
                if diag >= sum-diag && diag > 1e-25 {
                    p[row] = i
                    used[i] = true
                    if findPermutation(row + 1) {
                        return true
                    }
                    used[i] = false
                }
            }
        }
        return false
    }
    if findPermutation(0) {
```

```go
        newMatrix := make([][]float64, n)
        newB := make([]float64, n)
        for i := 0; i < n; i++ {
            newMatrix[i] = s.Matrix[p[i]]
            newB[i] = s.B[p[i]]
        }
        s.Matrix = newMatrix
        s.B = newB
        return true
    }
    return false
}
```

Листинг 2: Основной цикл метода простых итераций и расчет нормы

```go
func (s *IterativeSolver) Solve() SolvationResponse {
    if !s.CheckAndFixDominance() {
        return SolvationResponse{ErrorLog: "Diagonal dominance could not be achieved."}
    }
    n := s.N
    x := make([]float64, n)
    xPrev := make([]float64, n)
    var norm float64
    for i := 0; i < n; i++ {
        rowSum := 0.0
        for j := 0; j < n; j++ {
            if i != j {
                rowSum += math.Abs(s.Matrix[i][j] / s.Matrix[i][i])
            }
        }
        if rowSum > norm {
            norm = rowSum
        }
    }
    iters := 0
    maxIters := 10000
    for iters < maxIters {
        iters++
        copy(xPrev, x)
        maxDiff := 0.0
        for i := 0; i < n; i++ {
            sum := 0.0
            for j := 0; j < n; j++ {
                if i != j {
                    sum += s.Matrix[i][j] * xPrev[j]
                }
            }
            x[i] = (s.B[i] - sum) / s.Matrix[i][i]
            if math.IsInf(x[i], 0) || math.IsNaN(x[i]) {
                return SolvationResponse{ErrorLog: "The method differs (Infinity/NaN)."}
            }
            diff := math.Abs(x[i] - xPrev[i])
```

```go
            if diff > maxDiff {
                maxDiff = diff
            }
        }
        if maxDiff <= s.Eps {
            break
        }
    }
    rows := make([]ResultRow, n)
    for i := 0; i < n; i++ {
        rows[i] = ResultRow{
            Index: i + 1,
            X:     x[i],
            Error: math.Abs(x[i] - xPrev[i]),
        }
    }
    return SolvationResponse{Rows: rows, Iterations: iters, Norm: norm}
}
```

3.3 Фрагмент кода клиента (script.js)

Листинг 3: Валидация данных при чтении из файла

```js
document.getElementById('file-input').addEventListener('change', function(e) {
    const file = e.target.files[0];
    if (!file) return;
    const currentInput = e.target;
    const reader = new FileReader();
    reader.onload = function(event) {
        currentInput.value = '';
        const lines = event.target.result.split('\n')
            .map(l => l.trim())
            .filter(l => l.length > 0);
        if (lines.length < 2) return alert("The file is too short");
        const n = parseInt(lines[0]);
        if (isNaN(n) || n < 1 || n > 20 || lines[0] !== n.toString()) {
            return alert("Incorrect dimension n");
        }
        const validatedMatrix = [];
        const validatedB = [];
        for (let i = 0; i < n; i++) {
            if (!lines[i + 1]) return alert(`There are not enough lines.`);
            const rowValues = lines[i + 1].split(/\s+/).filter(v => v !== "");
            if (rowValues.length < n + 1) {
                return alert(`There is not enough data in the line ${i + 2}.`);
            }
```

```
            const matrixRow = [];
            for (let j = 0; j <= n; j++) {
                const valStr = rowValues[j].replace(',', '.');
                const num = parseFloat(valStr);
                if (isNaN(num) || !/^-?\d+(\.\d+)?([eE][+-]?\d+)?$
                   /.test(valStr)) {
                    return alert(`Error in line ${i + 2}: "${
                       rowValues[j]}" is not a number!`);
                }
                if (j < n) matrixRow.push(rowValues[j]);
                else validatedB.push(rowValues[j]);
            }
            validatedMatrix.push(matrixRow);
        }
        generateTable(n);
    };
    reader.readAsText(file);
});
```

Полный исходный код серверной части, включая обработку HTTP-запросов и маршрутизацию, содержится в файле `main.go`.

4 Примеры и результаты работы программы

Входные данные задаются размерностью n, матрицей коэффициентов A, вектором правых частей $\mathbf{b}$ и требуемой точностью ε. Программа выводит вектор решения $\mathbf{x}$, количество итераций, затраченных на достижение точности, норму матрицы перехода $\|C\|_\infty$ и вектор абсолютных погрешностей $|x_i^{(k)} - x_i^{(k-1)}|$ для каждой переменной.

4.1 Пример 1: Система 2×2 и перестановка строк

Рассмотрим систему, где в исходном виде диагональное преобладание отсутствует:

$$\begin{cases} 1 \cdot x_1 + 2 \cdot x_2 = 3, \\ 4 \cdot x_1 + 1 \cdot x_2 = 5. \end{cases}$$

Ввод: $n = 2$, матрица $\begin{pmatrix} 1 & 2 \\ 4 & 1 \end{pmatrix}$, вектор $\mathbf{b} = [3, 5]^T$, $\varepsilon = 0{,}001$.

Результат: Программа определяет отсутствие преобладания в первой строке ($|1| < |2|$) и автоматически производит перестановку строк. После перестановки система принимает вид:

$$\begin{cases} 4 \cdot x_1 + 1 \cdot x_2 = 5, \\ 1 \cdot x_1 + 2 \cdot x_2 = 3. \end{cases}$$

Здесь $|4| > |1|$ и $|2| > |1|$, условие выполнено. Метод простых итераций сходится

к решению $x_1 = 1$, $x_2 = 1$ за небольшое число итераций. Программа выводит норму матрицы перехода $\|C\|_\infty = 0{,}5$ и итоговые погрешности.

4.2 Пример 2: Работа с данными из файла (test.txt)

Для тестирования используется файл следующего содержания:

```
3
10 -1 2 11
-1 11 -1 9
2 -1 10 11
```

Здесь $n = 3$, далее следует расширенная матрица $[A|\mathbf{b}]$. Точность ε выставляется вручную, $\varepsilon = 0{,}0001$. В данной матрице выполнено строгое диагональное преобладание: $|10| > |-1| + |2|$, $|11| > |-1| + |-1|$, $|10| > |2| + |-1|$.

Результат работы программы:

1. **Норма матрицы перехода:** $\|C\|_\infty = 0{,}3$ (условие $\|C\| < 1$ соблюдено).
2. **Вектор неизвестных:** $x_1 = 0{,}958009$, $x_2 = 0{,}496186$, $x_3 = 0{,}958009$.
3. **Количество итераций:** 8.
4. **Вектор погрешностей:** значения $|x_i^{(k)} - x_i^{(k-1)}|$ для всех i не превышают 10^{-4}.

4.3 Пример 3: Невозможность достижения диагонального преобладания

Если ввести матрицу, в которой ни при каких перестановках строк нельзя добиться преобладания (например, все элементы равны или диагональные элементы равны нулю при наличии больших недиагональных коэффициентов), программа на этапе выполнения метода `CheckAndFixDominance` вернет ошибку.

Результат: В интерфейсе отобразится сообщение: «Ошибка: Не удалось достичь диагонального преобладания». Расчет при этом не производится, так как сходимость метода не гарантирована.

4.4 Пример 4: Особенности веб-интерфейса и валидация

Программа поддерживает ввод чисел с разделителем «запятая» (автоматически заменяется на «точку» в `script.js`). При попытке ввода некорректных данных (буквы, пустые поля) или размерности $n > 20$, система выдает предупреждение: «Ошибка при чтении таблицы. Проверьте, что все поля заполнены числами».

Также реализована функция «Случайные значения», которая генерирует матрицу с гарантированным диагональным преобладанием (диагональный элемент устанавливается как $\sum |a_{ij}| + 5$), что позволяет быстро тестировать работоспособность метода на больших размерностях.

5 Выводы

В ходе лабораторной работы была реализована программа решения СЛАУ методом простых итераций (Якоби). Численный метод реализован в виде структуры `IterativeSolver` на языке Go с передачей параметров через методы объекта, что обеспечивает инкапсуляцию логики решения. В программе реализованы ввод матрицы и вектора через веб-интерфейс (с поддержкой ручного ввода и загрузки из файла), ограничение размерности $n \leqslant 20$, расчет нормы матрицы перехода C, формирование вектора решения, подсчет количества итераций и вектора погрешностей.

Метод простых итераций базируется на использовании значений вектора неизвестных исключительно с предыдущего шага $\mathbf{x}^{(k)}$ для вычисления нового приближения $\mathbf{x}^{(k+1)}$. Здесь компоненты обновляются независимо, что позволяет четко отслеживать процесс сходимости. В работе реализован алгоритм автоматического поиска перестановки строк для достижения строгого диагонального преобладания; при невозможности его достижения программа выдает соответствующее предупреждение, так как сходимость метода в этом случае не гарантирована.

Вычисление и вывод строчной нормы матрицы перехода ($\|C\|_\infty$) позволяет верифицировать достаточное условие сходимости ($\|C\|_\infty < 1$). Информативный вывод вектора погрешностей по каждой компоненте подтверждает достижение заданной точности ε на последней итерации.

Программа успешно обрабатывает различные сценарии: системы со строгим диагональным преобладанием, системы, требующие перестановки строк, а также исключительные ситуации (недопустимая размерность, расходимость метода с получением значений типа `NaN` или `Inf`). Установленный лимит в 10 000 итераций предотвращает зацикливание при отсутствии сходимости.

Таким образом, цель работы полностью достигнута: реализован устойчивый алгоритм метода простых итераций, обеспечены гибкие способы взаимодействия с данными и корректное отображение всех диагностических величин.